

AI in IT operations

The emerging role of a decision model



AI is entering incident work, one capability at a time

Applications already interpret, coordinate and connect context; the decision is what remains

Interpret the incident

Observability platforms already interpret signals: Dynatrace's Davis correlates anomalies and points to a root cause, and Datadog's Bits AI SRE investigates incidents from operational evidence.

Coordinate the response

Service-management platforms coordinate the workflow: ServiceNow documents agents for alert assignment, analysis and verification.

Connect context to action

Cloud and network tooling connect context to response: AWS DevOps Agent draws on telemetry, code and deployment data, while F5 exposes device, VIP and pool context.



Each platform starts from a different position, and none completes the cycle

Selected examples; the categories overlap

Starting point	Examples	What it brings to the workflow
Service management	ServiceNow	Tickets, ownership, process; the broadest coverage
Observability	Dynatrace; Datadog	Signals, topology, service context
Cloud operations	AWS DevOps Agent	Infrastructure and cloud context
Specialist AI SRE	Resolve AI	Investigation and on-call work

ServiceNow covers the most of the cycle, but it can be expensive and slow to customise - which is where lighter, specialised components earn a place.

Think end to end: the cycle from Detect to Learn, and where custom components fit

A conceptual view of the workflow



Platforms and tools already automate detection, correlation, execution and record-keeping well. The links in the middle - interpretation, ownership, severity, and the act-or-page call - are where a small custom decision component can fit, without replacing anything around it.



The Jev opportunity: a decision model that fits those links

What it is, and why it is worth a look

What it is

TypeSafe's Jev is a 'System One' decision model, not a chat model. You send one request with a state - the alert and its context - and a set of typed questions, and it returns a typed answer with a probability on each. Three question types: Choice (pick from a list), Score (place on a scale) and Noul (the probability that a statement is true).

Why it is worth a look

The questions are yours and the thresholds are yours; ordinary code compares the probabilities with those thresholds and decides what happens. One call answers many questions in milliseconds, at a fraction of a cent of inference, and every call can be logged. Whether the answers are accurate enough on your tickets is the open question.



How this looks today, by hand - and with a decision step

The same incident, before and after

Today, by hand

An engineer reads the alert, opens the dashboards and the logs, looks for a recent change, judges how bad it is and whether to act, and pages whoever owns the service. It takes minutes to hours, and the answer depends on who is on call and how busy the shift is.

With a decision step

The platform sends the alert and its context to one call, which returns the classification, the severity and a proposed action, each with a probability. Ordinary code applies thresholds you own and either acts or asks a person - the same questions, answered the same way, every time.



An example, end to end: a service becomes unreachable

One recorded example from the project's set; the gate's checks with signed margins

INC-0007 · dns failure · AUTO

Check	Value / rule	Margin / result
Page a person	0.77 < 0.90	+0.13 / pass
Real incident	0.91 >= 0.50	+0.41 / pass
Action confidence	0.92 >= 0.60	+0.32 / pass
Action risk	0.50 <= 1.00	+0.50 / pass
Severity	2.35 <= 2.50	+0.15 / pass
Deploy correlation	0.94 >= 0.80	+0.14 / pass
Rollback score	0.79 >= 0.70	+0.09 / pass

A name-resolution failure is read once and classified as a network incident; the gate either permits an action or sends it to a person, with every check and its margin recorded.

Our measurement | one recorded example | gate checks from zeroops/policy.py



The same call, question by question

Twelve typed questions, answered in one request

This is one call. The state sent is the alert and its context - the service or VIP, recent changes, and the relevant log lines. The questions are fixed, and every answer comes back with a probability. The gate reads only the answers it needs.

Question	Answer
Is this a real incident?	0.91
Failure category	config
Suspected component	payment-svc
Likely root cause	dns_failure
Recent change implicated?	0.94
Severity (0-3)	2.35
Blast radius (0-3)	1.09
Customer impact?	0.71
Rollback safe?	0.79
Proposed first action	rollback
Action risk (0-2)	0.50
Page a person?	0.77



The same call, running end to end

The technical component: the contract, the primitives and the gate, in motion

The demo sends the same request through the contract, the three primitives and the gate. Nothing is scripted: the typed output and the gate decide each outcome, and the run ends on the same network incident used earlier.

AN EXAMPLE

Name resolution fails; the gate permits an action

Client

F5 VIP

Pool

DNS ✕

RESULT

root cause	dns_failure
needs_page	0.77 < 0.90
outcome	AUTO
executed	nothing

Embedded video [outputs/4_demo.mp4](#) 2:03 1080p



All four models worked; the differences are modest

The same questions, answered by four models

Backend	Exact action	Acceptable*	AUTO	Exact / AUTO	p50/p95 ms	API \$ / 1k
Jev (TypeSafe API)	28/56	52/56	22/56	13/22	91/128	\$0.057
Jev (via OpenRouter)	28/56	52/56	20/56	13/20	134/406	\$0.057
Kev-4B (local GPU)	39/56	48/56	8/56	8/8	589/609	GPU uncosted
Laya (local GPU)	11/56	19/56	0/56	—	91/128	GPU uncosted

*Acceptable-action criteria were defined after the first run; use exact action match as the primary measure.

Every model answered sensibly, and none automated a case labelled unsafe. The point is that a typed decision step is workable - not which model wins.

Our measurement | [work/results/facts.json](#) | corpus 0b4839761e06

How it integrates with the software you already run



One call from the tools in place; nothing is replaced

Existing tools

F5 · Dynatrace · ServiceNow
alerts, tickets, context



Jev

Typed questions; answers with
probabilities



Policy in your code

Thresholds you own decide act,
ask, or log

Possible responses within the workflow

Assign or prioritise

Investigate further

Consider a runbook

Documented integration points exist for common COTS software: ServiceNow (IntegrationHub REST step or MID Server script), PagerDuty (Event Orchestration rules, Custom Event Transformers, webhooks) and F5 (iRules or telemetry). A fixed flow acts on the answers and falls back to existing rules on timeout or low confidence.



Why a decision model is attractive: versatile, easy to configure, low cost

The properties to weigh against the platform's own AI

Versatile

One interface covers classification, severity, routing and yes/no checks, across application and network incidents.

Easy to configure

Questions and thresholds are written in code - no prompt engineering and no model training to start.

Low cost

A fraction of a cent of inference per incident; the real cost is integration and ownership.

Fast and owned

Milliseconds per call, and ordinary code holds the thresholds and the decision.

Flexibility and cost sit on one side; data handling, integration and the effort of ownership sit on the other. That is the trade to weigh against what the platform already provides.



How to explore this together: size the opportunity, then a staged pilot

Three short stages, each ending in a decision

Stage 1

Diagnostic (2-3 weeks)

Baseline reassignment, triage and out-of-hours volume; what the current licence already does.

DECISION 1

Is there a gap worth closing?

Stage 2

Offline replay (3-4 weeks)

2,000-5,000 closed tickets through the same questions, compared with current rules; thresholds set on held-out tickets.

DECISION 2

Does it beat what runs today, by enough to matter?

Stage 3

Shadow, then assist (6-8 weeks)

Answers to hidden fields with no action, then suggestions with override tracking, inside the platform.

DECISION 3

What autonomy, under which owner?

Where outside help fits: bring this harness, the question design and the gate; baseline your own data; work alongside your platform and data teams; support the compliance and operating-model calls.

Appendix

Detail behind the discussion



Full results across four backends

All four answered the same 12 questions about the same 56 incidents

Backend	Exact action	Acceptable*	AUTO	Exact / AUTO	p50/p95 ms	API \$ / 1k
Jev (TypeSafe API)	28/56	52/56	22/56	13/22	91/128	\$0.057
Jev (via OpenRouter)	28/56	52/56	20/56	13/20	134/406	\$0.057
Kev-4B (local GPU)	39/56	48/56	8/56	8/8	589/609	GPU uncosted
Laya (local GPU)	11/56	19/56	0/56	—	91/128	GPU uncosted

*Acceptable-action criteria were defined after the first run; use exact action match as the primary measure.

Exact action match is the primary measure; the acceptable-action column uses a list defined after the first run and is shown for completeness.

Our measurement | [work/results/facts.json](#) | corpus 0b4839761e06



Twelve considerations that decide whether this fits an estate

Short titles; full text in `outputs/3_open-questions-and-next-steps.md`

#	Consideration
C1	It fits text classification at volume, and little else
C2	The value is the gap to what you already run
C3	Results depend on what the ticket says at the moment of decision
C4	Thresholds and calibration must come from your own data
C5	Question wording and option order shape the answer
C6	Anyone who can write into a ticket can lean on the answer
C7	The gate is only as good as what it checks and how it fails
C8	The vendor is young, hosted in one region, and the model moves
C9	The API call is the easy part of the integration
C10	Inference is almost free; people and alternatives decide the economics
C11	Regulatory duties depend on how each use is classified
C12	Someone has to own the questions, the thresholds and on-call trust



Where a typed model fits, component by component

A judgement view from the project's research

Component	Build vs buy	Where a typed model fits
Ingestion / observability	Build or buy	Not applicable
Anomaly detection	Buy or keep - numeric	No fit (weak at numbers)
Correlation / dedup	Hybrid	Supplement: 'same incident?' after dedup
CMDB / topology	Keep the system of record	Supplement: map text to CI
Root cause analysis	Buy topology RCA or pilot an agent	Supplement: rank candidates
Triage / routing	Strongest build candidate	Primary use
Runbooks / remediation	Build is viable	Gate only, never authority



Risks, security and compliance, and how each could be contained

The conditions a real test would manage

Risk	Containment
Adversarial text in tickets or logs can move an answer	Keep untrusted text out of questions that gate actions; keep action authority outside the model
First-option bias (16 vs 4)	Shuffle option order and repeat; set thresholds on held-out data
Thresholds and calibration are unproven on our data	Choose thresholds on held-out tickets; re-run after each model release
Vendor is young; model moves under an alias; no EU region	Pin the version; redaction and a signed DPA; start in shadow mode
Regulatory duties depend on classification	Log input hash, model version, option order, probabilities and threshold version; take legal review



Open questions by owner

A selection; all 23 in outputs/3_open-questions-and-next-steps.md

Owner	Questions
SRE / operations	Reassignment rate and cost (Q1); text-only decisions (Q2); ticket quality (Q3); novel root causes (Q4); existing automated fixes (Q5); error-cost ratio (Q6); trust in suggestions (Q7)
ITSM / platform	What the current licence already does (Q8); assignment-group separability (Q9)
Security / risk / procurement / finance / leadership	Data handling (Q12, Q14); threshold ownership (Q22, Q23); economics (Q19); where the decision sits (Q7)



Open or proprietary: both work; the trade-off is control against convenience

The decision layer can be hosted or run in your environment

Hosted (for example Jev)

Adopt quickly and operate nothing; the trade-off is that incident text leaves your environment, under a signed DPA and zero-retention terms.

Open (for example Laya or Kev)

Run it inside your network with no data egress; you own the GPUs, fine-tuning and upkeep, and untuned accuracy starts lower.

Both work well

Either can serve as the decision layer. This project's harness runs any model, so the choice can follow data-residency and operating preferences.



About Jev and TypeSafe

What is known about the model used as the example, as of October 2026

Item	Detail
Vendor	TypeSafe AI, San Francisco; Jev entered early access in September 2026
Model	Jev, marketed as a 'System One' decision model: not a chat model, and it does not generate text
Interface	One REST call: a state in; typed answers with probabilities out
Primitives	Choice (pick from a list), Score (place on a scale), Noul (probability a statement is true)
Hosting and price	Hosted API in one region; input tokens billed, output free (list about \$0.042 per million input tokens)
Status	Young and moving; this project pins the version (jev-1.13.0) and re-validates after each release



Sources and basis of the discussion

Primary vendor material checked 4 October 2026; project results for the example figures

S1**[ServiceNow · ITOM agentic workflows](#)**

Linked primary source

S2**[Datadog · Building Bits AI SRE](#)**

Linked primary source

S3**[AWS · DevOps Agent general availability](#)**

Linked primary source

S4**[Dynatrace · Multicloud SRE agents](#)**

Linked primary source

S5**[Resolve AI · AI SRE platform](#)**

Linked primary source

S6**[TypeSafe · Jev introduction](#)**

Linked primary source

S7**[TypeSafe · API reference](#)**

Linked primary source

Example figures: saved facts and the results report; corpus 0b4839761e06